

Faculty of Engineering and Technology Electrical and Computer Engineering Department

Hardware Design Lab

Timer DUT Verification Report

Prepared by: Besan Maaly 1222776

Instructor: Dr. Abdellatif Abu-Issa

Teaching Assistant : Jehad Halahla

September 2026

Contents

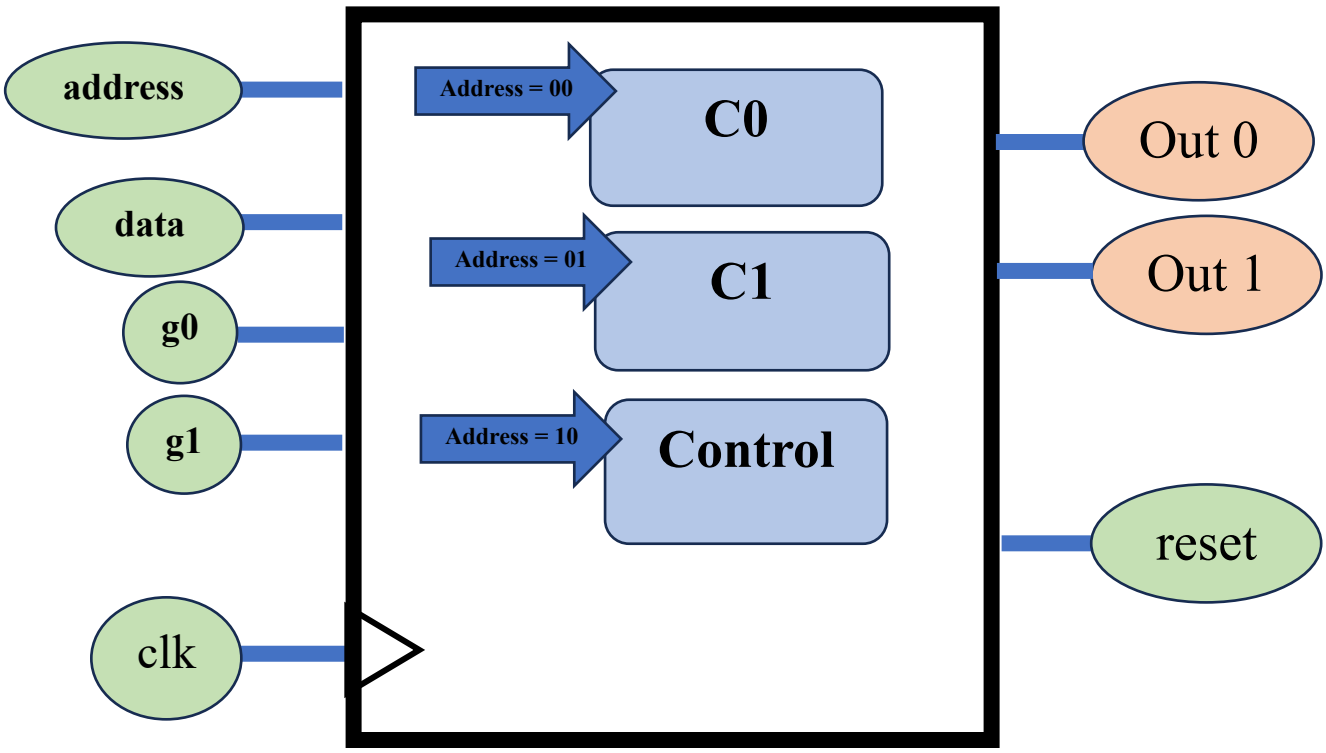
1. Introduction :..... 2
2. Specification 3
2.1. Timer Interface :..... 3
2.2. Register Programming 4
2.3 Output Modes..... 4
3. Verification Environment..... 5
3.1. Test Bench Architecture 5
3.2. Reference Model and Scoreboard..... 6
3.3. Functional Coverage Model..... 7
3.4. SVA Assertions..... 7
4.Test Plan..... 8
5. Corner Cases Covered..... 10
6.Functional Coverage 11
6.1. Coverage Points and Crosses 11
6.2. Coverage results..... 13
7. Bugs Found 14
7.1. Reset Addition..... 15
7.2. Invalid Address Behavior..... 15
7.3. Interpretation of Test Results 15
8.How to Reproduce 16
9. Conclusion 17

1. Introduction :

This report presents a pure System Verilog verification environment for a programmable dual timer. The design exposes two 8-bit counter registers, one 8-bit control register, a 2-bit address bus, a 4- bit bidirectional data bus, two gate inputs, a clock, and two divided-clock outputs. The verification strategy combines constrained random testing, directed corner case tests, a reference model, a scoreboard, assertions, and functional coverage. The verification environment is intended to detect violations of the specified programming protocol, counter ranges, mode restrictions, gate behavior, output timing, invalid-address behavior, and dynamic mode changes. The supplied RTL is treated as intentionally buggy. The objective is to expose and document the bugs rather than repair the RTL. The verification campaign focuses on detecting missing counter-range and mode-parity validation, incorrect Counter 0 write-gating behavior, improper handling of incomplete writes, and output equations that do not match the specified mode behavior. These issues are analyzed using the reference model, scoreboard results, assertions, and directed tests.

2. Specification

2.1. Timer Interface :



Signal	Width	Purpose
Address	2 bits input	00 = counter0 01 = counter1 10 = control 11 = invalid
Data	4 bits in/out	Data bus. Two transfers are required to program each 8-bit counter: low bits first, then high bits
g0	1 bit input	Gate for counter0 when g0 = 1 ➔advances when g0 = 0 ➔ freezes
g1	1 bit input	Gate for counter1 when g1 = 1 ➔advances when g1 = 0 ➔ freezes
Clk	1 bit input	Register writes and counter updates occur on posedge clk
Out 0	1 bit output	Divided output clock generated by counter0 according to mode0
Out 1	1 bit output	Divided output clock generated by counter1 according to mode1
Reset	1 bit input	Asynchronous active low reset

2.2. Register Programming

The counter registers are 8 bits wide, while the data bus is 4 bits wide. Therefore, the low bit group is written first and the high bit group is written second. The testbench must check that an incomplete write does not update the effective counter value.

The control write uses the low four data bits. Bit d[3] selects the counter, where zero selects counter 0 and one selects counter 1. Bits d[2:0] select the mode. The upper four bits are specified as dont care bits and should not affect the operation.

Address 11 is invalid. A transaction to this address must not change any design state. An odd value is illegal for mode 2, and the mode must remain unchanged when such a value is attempted. Counter 0 accepts values from 50 through 200 inclusive. Counter 1 accepts values from 2 through 150 inclusive.

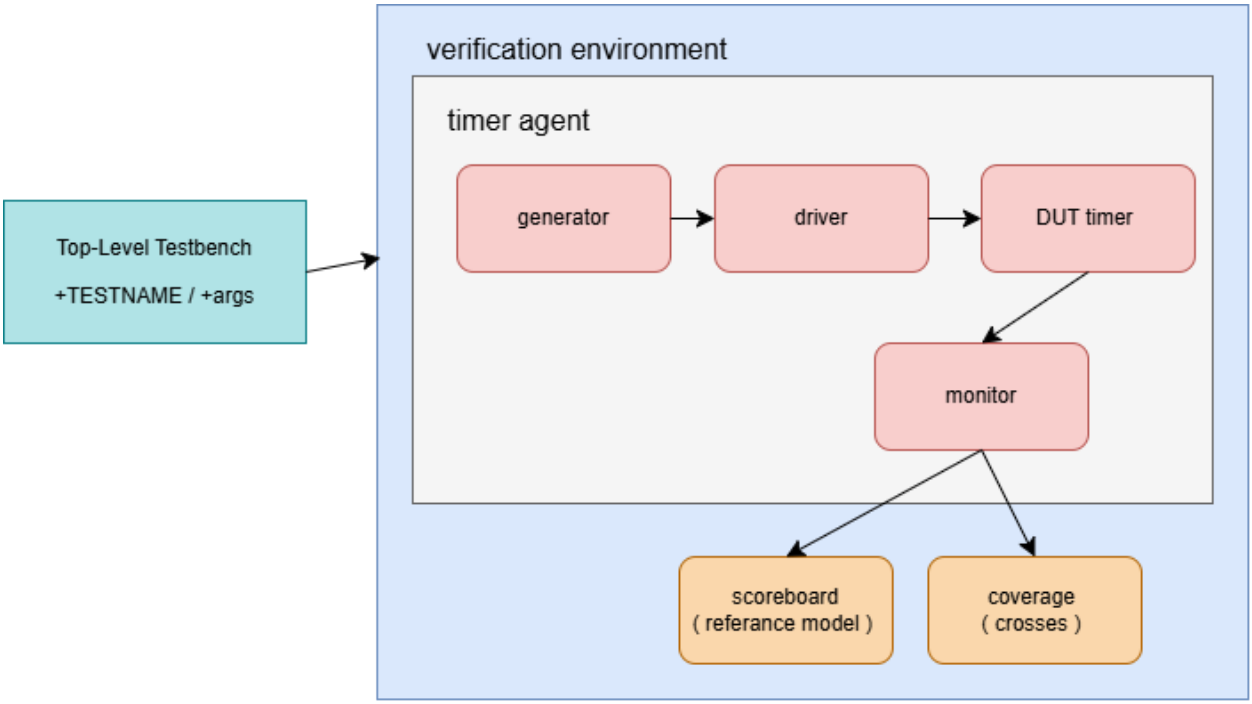
2.3 Output Modes

For a programmed count value n, the required behavior is:

Mode	Required pattern	Constraint
0	One high pulse and n-1 low pulses; duty cycle 1/n	No additional parity restriction
1	n-1 high pulses and one low pulse; duty cycle (n-1)/n	No additional parity restriction
2	n/2 low pulses and n/2 high pulses; duty cycle 1/2	n must be even
3	n+1/2 low pulses and n-1/2 high pulses	n must be odd
4	n-1/2 low pulses and n+1/2 high pulses	n must be odd

3. Verification Environment

The environment is a pure System Verilog, class-based testbench. It contains a transaction object, a generator, a driver, a monitor, a reference model, a scoreboard, functional coverage, an agent, an environment, and a top-level testbench. A virtual interface connects the class-based verification components to the DUT signals. Runtime test selection and configuration are supported through plusargs.



3.1. Test Bench Architecture

The environment is class based and implemented using pure System Verilog without any UVM library. It follows the classic generator/driver/monitor/scoreboard architecture:

- **timer_if** : The System Verilog interface connects the testbench to the DUT signals. The driver accesses the interface through a virtual interface and controls bus timing using clock-edge synchronization.
- **timer_txn / bus_txn** :Transaction objects used to represent stimulus and monitored bus activity. Transactions are generated by the generator and driven to the DUT, while monitored transactions are forwarded to the scoreboard and coverage components.
- **timer_generator** : Generates stimulus using reusable tasks such as counter writes, control writes, invalid-address transactions, idle cycles, reset pulses, and dedicated scenarios for the different tests.

- **timer_driver** : Acts as the bus functional model (BFM). It converts generated transactions into pin-level activity through the virtual interface and clock-edge synchronization.
- **timer_monitor** : A passive monitor that samples the DUT interface and forwards bus snapshots to both the scoreboard and functional coverage.
- **timer_ref_model** :The reference model (golden model) calculates the expected counter/register behavior and expected out0/out1 values based on the specification, independently from the RTL implementation.
- **timer_scoreboard** :Compares the DUT outputs against the expected values generated by the reference model and reports mismatches.
- **timer_coverage** :Implements functional coverage for important verification scenarios, including address values, operating modes, counter ranges/corner cases, gate and reset behavior, and relevant cross coverage.
- **timer_agent / timer_env** :The agent groups the generator, driver, and monitor. The environment then integrates the agent with the scoreboard and coverage components.
- **base_test + 10 concrete tests** :Provides multiple directed, corner-case, and randomized tests. Tests can be selected and configured at run time using +TESTNAME=<name> and other plusargs without recompiling the verification environment.

3.2. Reference Model and Scoreboard

The timer_ref_model implements the expected behavior of the timer independently from the RTL. It stores the accepted counter values, operating modes, programming pointers, counter phases, and partial low bits writes.

For every monitored clock cycle, the reference model processes the address, data, gate, and reset signals. A counter write is accepted only after two bit groups are received and the complete value satisfies the corresponding counter range and mode-parity requirements. The model also checks the Counter 0 write-gating rule and ignores invalid address transactions.

The reference model calculates the expected out0 and out1 values according to the five specified operating modes. The timer_scoreboard compares these expected values and internal states with the observed DUT values. It reports mismatches in:

- out0 and out1
- Counter 0 and Counter 1 registers
- Mode 0 and Mode 1 registers
- Counter phases cnt0 and cnt1
- Total state mismatches

The scoreboard result is reported as PASS only when no output or state mismatch is detected.

3.3. Functional Coverage Model

Functional coverage is implemented in the timer_coverage class using the cg_bus covergroup. The coverage model observes the monitored bus transactions, DUT modes, attempted counter values, gate signals, reset state, and incomplete writes.

The main coverpoints are:

- Address values 00, 01, 10, and 11
- Modes 0–4 and unsupported modes 5–7
- Counter 0 value categories
- Counter 1 value categories
- Both states of g0 and g1
- Reset asserted and released
- Complete and incomplete writes for both counters

The coverage model also includes the following crosses:

- Address × g0
- Address × g1
- Mode 0 × Counter 0 value category
- Mode 1 × Counter 1 value category
- Mode 0 × g0
- Mode 1 × g1

This coverage model measures whether the specified input scenarios and their important combinations were exercised. Coverage completion does not imply that the DUT passed the scoreboard checks.

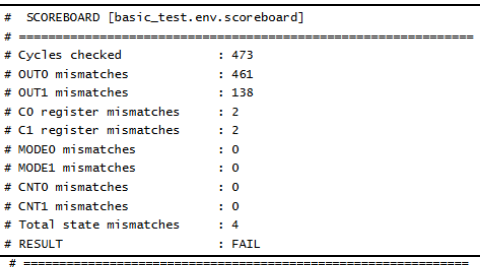
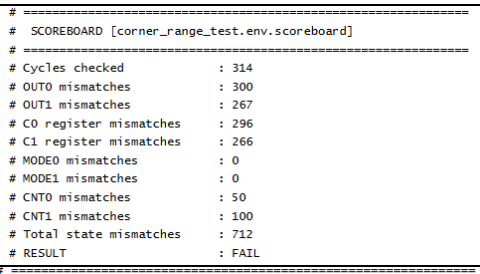
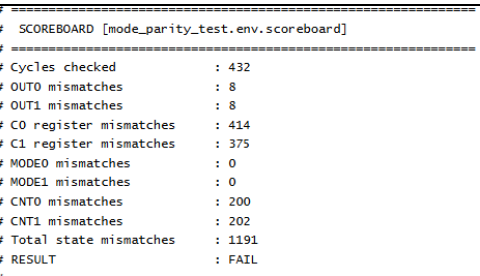
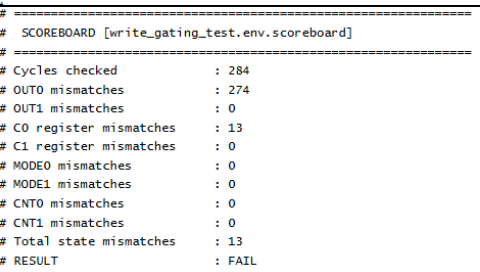
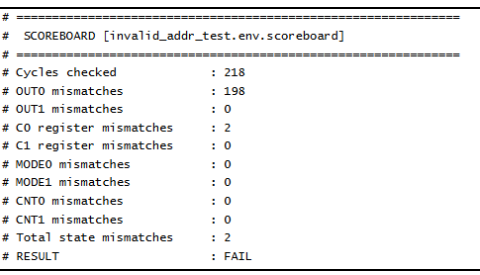
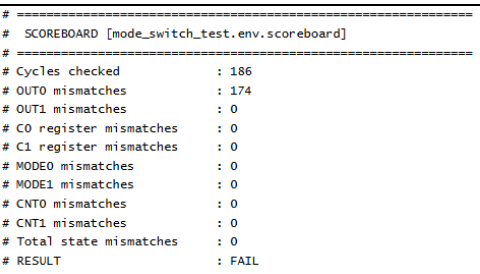
3.4. SVA Assertions

The timer_sva_checker module contains concurrent assertions for important safety requirements. The assertions are sampled on the rising edge of clk.The implemented properties check that:

1. Reset clears the counters, modes, programming pointers, and counter phases.
2. Counter 0 phase remains stable when g0=0.
3. Counter 1 phase remains stable when g1=0.
4. An invalid address does not modify the programmed counters or modes.
5. A Counter 0 write is blocked while g0=1.
6. The timer outputs do not become unknown after reset is released.

These assertions complement the reference-model-based scoreboard. The scoreboard checks expected functional behavior, while the assertions continuously monitor specific safety properties during simulation.

4.Test Plan

Test	Purpose	Result	screenshot
basic_test	Smoke test: program counter0/mode0 and counter1/mode1, let each free run a few periods.	FAIL	
corner_range_test	Sweeps every boundary of the legal ranges (49/50/51/199/200/201 for counter0, 1/2/3/149/150/151 for counter1).	FAIL	
mode_parity_test	For mode2/3/4, writes both a parity-legal and a parity-illegal count value and checks the illegal one is rejected.	FAIL	
write_gating_test	Writes to counter0 both while g0=1 (counting) and g0=0 (idle) and checks the write is blocked/allowed accordingly.	FAIL	
invalid_addr_test	Issues address 2'b11 and checks that no programmed counter or mode register is modified	FAIL	
mode_switch_test	Switches mode0 to mode1 mid count (observational).	FAIL	

reset_test	Programs state, then pulses rst_n and checks everything returns to the reset default.	FAIL	# == # SCOREBOARD [reset_test.env.scoreboard] # == # Cycles checked : 144 # OUT0 mismatches : 26 # OUT1 mismatches : 0 # C0 register mismatches : 2 # C1 register mismatches : 0 # MODE0 mismatches : 0 # MODE1 mismatches : 0 # CNT0 mismatches : 0 # CNT1 mismatches : 0 # Total state mismatches : 2 # RESULT : FAIL
random_regression_test	Weighted-random mix of every operation type, NUM_TXNS configurable general soak/regression.	FAIL	# == # SCOREBOARD [random_regression_test.env.scoreboard] # == # Cycles checked : 2261 # OUT0 mismatches : 638 # OUT1 mismatches : 944 # C0 register mismatches : 2054 # C1 register mismatches : 1575 # MODE0 mismatches : 0 # MODE1 mismatches : 0 # CNT0 mismatches : 2073 # CNT1 mismatches : 1388 # Total state mismatches : 7090 # RESULT : FAIL
incomplete_write_test	Sends only the low bits of a counter value, verifies that the counter is not committed prematurely, and then performs a complete write.	FAIL	# == # SCOREBOARD [incomplete_write_test.env.scoreboard] # == # Cycles checked : 101 # OUT0 mismatches : 93 # OUT1 mismatches : 0 # C0 register mismatches : 77 # C1 register mismatches : 0 # MODE0 mismatches : 0 # MODE1 mismatches : 0 # CNT0 mismatches : 0 # CNT1 mismatches : 0 # Total state mismatches : 77 # RESULT : FAIL
output_mode_test	Exercises output modes 0-4 on both counters and observes the generated divided clock duty cycles.	FAIL	# == # SCOREBOARD [output_mode_test.env.scoreboard] # == # Cycles checked : 1278 # OUT0 mismatches : 304 # OUT1 mismatches : 183 # C0 register mismatches : 235 # C1 register mismatches : 162 # MODE0 mismatches : 0 # MODE1 mismatches : 0 # CNT0 mismatches : 750 # CNT1 mismatches : 150 # Total state mismatches : 1297 # RESULT : FAIL
all_tests_test	Runs all directed scenarios and the randomized regression in one combined simulation.	FAIL	# == # SCOREBOARD [all_tests_test.env.scoreboard] # == # Cycles checked : 10505 # OUT0 mismatches : 3339 # OUT1 mismatches : 3198 # C0 register mismatches : 8243 # C1 register mismatches : 6286 # MODE0 mismatches : 0 # MODE1 mismatches : 0 # CNT0 mismatches : 8552 # CNT1 mismatches : 6006 # Total state mismatches : 29087 # RESULT : FAIL

5. Corner Cases Covered

- **Counter0 legal-range boundaries**: attempted values 49, 50, 51, 199, 200, and 201, where the legal range is 50-200 inclusive.
- **Counter1 legal-range boundaries**: attempted values 1, 2, 3, 149, 150, and 151, where the legal range is 2-150 inclusive.
- **Counter0 parity restrictions:** mode2 was tested with an even legal value and an odd illegal value. Modes3 and 4 were tested with an odd legal value and an even illegal value.
- **Counter1 parity restrictions:** mode2 was tested with an even legal value and an odd illegal value. Modes3 and 4 were tested with an odd legal value and an even illegal value.
- **Two bit group counter programming**: each complete counter write was performed using two consecutive 4-bit transfers. The low bit group was written first, followed by the high bit group.
- **Incomplete counter writes**: a low bit group only write was issued for counter0 and counter1. The reference model checks that the counter value is not committed before the high bit group is received.
- **Invalid address handling**: address 2'b11 was exercised as an invalid address. The expected behavior is that it must not modify the programmed counter or mode registers. Counter phase progression may continue independently when the corresponding gates are enabled.
- **Write gating for counter0**: counter0 writes were attempted while g0=1 and while g0=0. The expected behavior is that a write is blocked while the counter is counting and allowed while the counter is stopped.
- **Write gating for counter1**: counter1 was also exercised with both gate states to verify the corresponding freeze and write behavior.
- **Gate freeze behavior**: both counters were tested with their corresponding gate signal equal to zero. The count state is expected to remain unchanged until the gate returns to one.
- **Free running behavior**: both counters were allowed to run with their corresponding gate signal equal to one, allowing the divided output behavior to be observed over multiple cycles.
- **All output modes**: modes 0, 1, 2, 3, and 4 were exercised on both counters.
- **Mode dependent output behavior**: the output duty-cycle behavior was observed for the one-high-pulse, one-low-pulse, half-duty-cycle, and odd-count modes specified by the assignment.
- **Mode switching during operation**: counter0 was allowed to count in mode0, after which the control register was changed to mode1 while counting was still in progress. The resulting output and counter phase were compared with the reference model.

- **Reset during operation**: reset was asserted after the timer had been programmed and allowed to count. The reset behavior was checked for counters, modes, pointers, and count state.
- **Reset during a programming sequence**: reset was also exercised around counter programming activity to verify that partially programmed state does not remain active after reset.
- **Reset assertion and release**: both reset-asserted and reset-released states were sampled by the functional coverage model.
- **Control register selection**: control writes were performed for both counter0 and counter1 using the control-word counter-select bit.
- **Unsupported mode values**: randomized control transactions can exercise mode values outside the supported range 0-4. These values are monitored through the illegal-mode coverage bins.
- **Simultaneous gate combinations**: combinations of g0 and g1 were exercised, including both counters enabled, both counters disabled, and one counter enabled while the other is stopped.
- **Randomized corner interaction**: constrained-random regression was used to combine addresses, modes, counter values, gates, reset activity, and operation types in different sequences. This helps expose interactions that may not be detected by isolated directed tests.

6.Functional Coverage

6.1. Coverage Points and Crosses

The functional coverage model was implemented in the timer_coverage class using the cg_bus covergroup. The coverage model observes the address, programmed modes, attempted counter values, gate signals, reset behavior, incomplete writes, and important cross-coverage combinations.

The coverage model includes the following cover points and crosses:

1. cp_addr: Covers all four address values:
   - 2'b00: counter0
   - 2'b01: counter1
   - 2'b10: control register
   - 2'b11: invalid address
2. cp_mode0: Covers modes 0–4 for counter0, in addition to unsupported mode values 5–7.
3. cp_mode1: Covers modes 0–4 for counter1, in addition to unsupported mode values 5–7.
4. cp_attempt_c0: Covers attempted counter0 values divided into:

- Below minimum
- At minimum
- Near minimum
- Middle range
- Near maximum
- At maximum
- Above maximum

5. cp_attempt_c1: Covers attempted counter1 values divided into:
   - Below minimum
   - At minimum
   - Near minimum
   - Middle range
   - Near maximum
   - At maximum
   - Above maximum
6. cp_g0: Covers both gate states for counter0:
   - g0 = 0
   - g0 = 1
7. cp_g1: Covers both gate states for counter1:
   - g1 = 0
   - g1 = 1
8. cp_rst: Covers both reset states:
   - Reset asserted
   - Reset released
9. cp_incomplete_c0: Covers complete and incomplete counter0 write transactions.
10. cp_incomplete_c1: Covers complete and incomplete counter1 write transactions.
11. cx_addr_g0: Cross coverage between all address values and both g0 states.
12. cx_addr_g1: Cross coverage between all address values and both g1 states.
13. cx_mode0_c0: Cross coverage between counter0 modes and attempted counter0 value categories.
14. cx_mode1_c1: Cross coverage between counter1 modes and attempted counter1 value categories.
15. cx_mode0_gate: Cross coverage between counter0 modes and both g0 gate states.
16. cx_mode1_gate: Cross coverage between counter1 modes and both g1 gate states.

6.2. Coverage results

```
# ============================================================
#  FUNCTIONAL COVERAGE [all_tests_test.env.coverage]
# ============================================================
# Address coverage          : 100.00%
# Mode0 coverage            : 100.00%
# Mode1 coverage            : 100.00%
# C0 attempted values       : 100.00%
# C1 attempted values       : 100.00%
# G0 coverage               : 100.00%
# G1 coverage               : 100.00%
# Reset coverage            : 100.00%
# Incomplete C0 coverage    : 100.00%
# Incomplete C1 coverage    : 100.00%
# Address x G0              : 100.00%
# Address x G1              : 100.00%
# Mode0 x C0 value          : 54.76%
# Mode1 x C1 value          : 61.90%
# Mode0 x G0                : 100.00%
# Mode1 x G1                : 100.00%
# OVERALL                   : 94.79%
# ============================================================
```

The following results were obtained from the combined all_tests_test regression:

Coverage Point	**Coverage**	**Description**
cp_addr	100.00%	All valid and invalid address values were exercised.
cp_mode0	100.00%	Modes 0–4 and unsupported mode values were covered.
cp_mode1	100.00%	Modes 0–4 and unsupported mode values were covered.
cp_attempt_c0	100.00%	All counter0 value categories were attempted.
cp_attempt_c1	100.00%	All counter1 value categories were attempted.
cp_g0	100.00%	Both gate states were covered.
cp_g1	100.00%	Both gate states were covered.
cp_rst	100.00%	Reset asserted and reset released states were covered.
cp_incomplete_c0	100.00%	Complete and incomplete counter0 writes were exercised.
cp_incomplete_c1	100.00%	Complete and incomplete counter1 writes were exercised.
cx_addr_g0	100.00%	All address and g0 combinations were covered.
cx_addr_g1	100.00%	All address and g1 combinations were covered.
cx_mode0_c0	54.76%	Some mode0 and counter0-value combinations remained uncovered.
cx_mode1_c1	61.90%	Some mode1 and counter1-value combinations remained uncovered.
cx_mode0_gate	100.00%	All mode0 and g0 combinations were covered.
cx_mode1_gate	100.00%	All mode1 and g1 combinations were covered.
Overall(cg_bus)	**94.79%**	Overall functional coverage for the combined regression.

The lower coverage of the mode/value crosses is expected because not every combination is legal. For example, Mode 2 requires an even counter value, while Modes 3 and 4 require an odd counter value. Therefore, some cross bins cannot be reached by legal transactions.The overall coverage of **94.79%** indicates that most planned scenarios were exercised. It does not indicate that the DUT passed. The scoreboard results still show mismatches between the intentionally buggy RTL and the independent reference mode

7. Bugs Found

Bug	the Reason	Detected By
No range validation on counter writes	The RTL directly writes the low and high bit groups into the counter registers. It does not check the legal range of Counter 0 (50–200) or Counter 1 (2–150).	corner_range_test all_tests_test
No mode/parity validation on counter writes	The RTL does not reject an odd value in Mode 2 or an even value in Modes 3 and 4. The counter write path has no check against the active mode requirements.	mode_parity_test all_tests_test
Counter 0 writes are not blocked while counting	The specification requires a Counter 0 write to be blocked when g0=1. The RTL address-decode logic does not check g0 before accepting the write.	write_gating_test
Incomplete counter writes modify the effective counter register	After receiving only the low bit group ,the RTL immediately updates part of the counter register. According to the specification, an incomplete write must not update the effective counter value until the high bit group is received and the complete value is validated.	incomplete_write_test
Mode 0 output equation does not implement the specified duty cycle	The specification requires one High pulse and n-1 Low pulses. The RTL uses cnt < counter - 2, which does not implement the required Mode 0 pattern.	output_mode_test basic_test all_tests_test
Mode 1 output equation does not implement the specified duty cycle	The specification requires n-1 High pulses and one Low pulse. The RTL uses cnt < 1, which does not implement the required Mode 1 pattern.	output_mode_test basic_test all_tests_test
Mode 2 uses division by 3 instead of division by 2	The RTL uses counter / 3 in the Mode 2 output equation. The specification requires equal Low and High durations based on n/2.	mode_parity_test output_mode_test all_tests_test
Mode 3 uses n/2 instead of (n+1)/2	The required Mode 3 pattern is (n+1)/2 Low pulses followed by (n-1)/2 High pulses. The RTL uses counter / 2, which is incorrect for odd values of n.	mode_parity_test output_mode_test all_tests_test
Mode 4 uses (n-3)/2 instead of (n-1)/2	The RTL subtracts 3 before division, while the specification requires subtraction of 1.	mode_parity_test output_mode_test all_tests_test
Unsupported mode values 5–7 are accepted	The control-register logic writes any value of d[2:0] into the mode register. There is no check that the mode is within the legal range 0–4.	random_regression_test all_tests_test,

7.1. Reset Addition

The active low asynchronous reset rst_n was added because it is explicitly required by specification video. This addition is not considered a functional bug fix.

The reset behavior is checked by reset_test, the ap_reset_state assertion, and the reset handling in the reference model and scoreboard. The FAIL result of reset_test indicates that the complete test produced scoreboard mismatches; it does not mean that the required reset addition was incorrect.

7.2. Invalid Address Behavior

Address 2'b11 is handled by the RTL as default.

Therefore, an invalid address does not intentionally modify the programmed registers, and this behavior is not classified as an RTL bug. It is checked by invalid_addr_test, the ap_invalid_addr assertion, and the reference model.

The test may still report FAIL because the counters can continue advancing while the gates are enabled. Consequently, a counter-phase or output mismatch during an invalid-address cycle must not automatically be attributed to an invalid-address programming error.

7.3. Interpretation of Test Results

All directed tests in the report ended with FAIL

These failures result from mismatches between the DUT and the independent reference model. The screenshots show mismatches in outputs, counter registers, modes, and counter phases, depending on the test scenario.

The main evidence is summarized below:

- corner_range_test: mismatches while testing legal and illegal counter boundary values.
- mode_parity_test: mismatches while testing legal and illegal parity combinations.
- write_gating_test: Counter 0 register mismatches when writing while g0=1.
- incomplete_write_test: mismatches after a low bit group only write.
- output_mode_test: output mismatches while exercising Modes 0–4.
- all_tests_test: combined directed and constrained-random regression with the largest number of mismatches.

The overall functional coverage reached 94.79%. This indicates that most verification scenarios were exercised; it does not indicate that the DUT passed. The scoreboard failures demonstrate mismatches between the DUT and the reference model. However, each individual bug must be attributed only when the corresponding mismatch is directly related to the tested behavior.

8.How to Reproduce

The verification environment was run on EDA Playground using Siemens Questa 2025.2 , and The testbench supports runtime test selection through plusargs.

. The following options were entered in the Run Options field:

1. **-voptargs=+acc=npr** ➔ The option enables access to the DUT internal signals used by the monitor and scoreboard.
2. **+TESTNAME=<test_name>**➔ The argument runs the test
3. **+NUM_TXNS=<number_of_random_transactions>**➔ Specifies the number of transactions for random_regression_test.
4. **+TIMEOUT=<timeout_in_cycles>**➔ Sets the watchdog timeout in clock cycles

The coverage results were obtained using:

-voptargs=+acc=npr +TESTNAME=all_tests_test

The available tests are:

- basic_test
- corner_range_test
- mode_parity_test
- write_gating_test
- invalid_addr_test
- mode_switch_test
- reset_test
- random_regression_test
- incomplete_write_test
- output_mode_test
- all_tests_test

9. Conclusion

A pure System Verilog verification environment was developed for the programmable dual timer. The environment includes a transaction generator, driver, monitor, reference model, scoreboard, functional coverage model, assertions, directed tests, and constrained-random regression.

The verification tests exercised the main requirements of the specification, including counter-range boundaries, two bit group programming, incomplete writes, mode restrictions, gate behavior, invalid addresses, reset behavior, mode switching, output modes, and randomized operation.

The combined regression achieved an overall functional coverage of 94.79%. Most individual cover points and basic crosses reached 100%, while the remaining uncovered bins were mainly in the mode/value crosses.

The scoreboard reported failures because the supplied RTL is intentionally buggy and does not fully match the specification. The main issues identified are missing range and parity validation, missing Counter 0 write gating, incorrect incomplete-write behavior, and incorrect output equations for several modes.

The active-low asynchronous reset was added as required by the specification and is not classified as a functional bug. The invalid address 2'b11 is implemented as a no-operation and is also not classified as a programming-state bug.

Overall, the verification environment successfully exercised the required behaviors, measured functional coverage, and exposed differences between the DUT implementation and the specification without modifying the intentionally buggy functional logic.